

Servlet Notes

There are two types of servers:

1. Web Server
2. Application server

Understanding Servers

Think of a server as a specialized computer that stays on 24/7 to serve requests from users. Based on what they can handle, we divide them into two main categories:

1. Web Servers

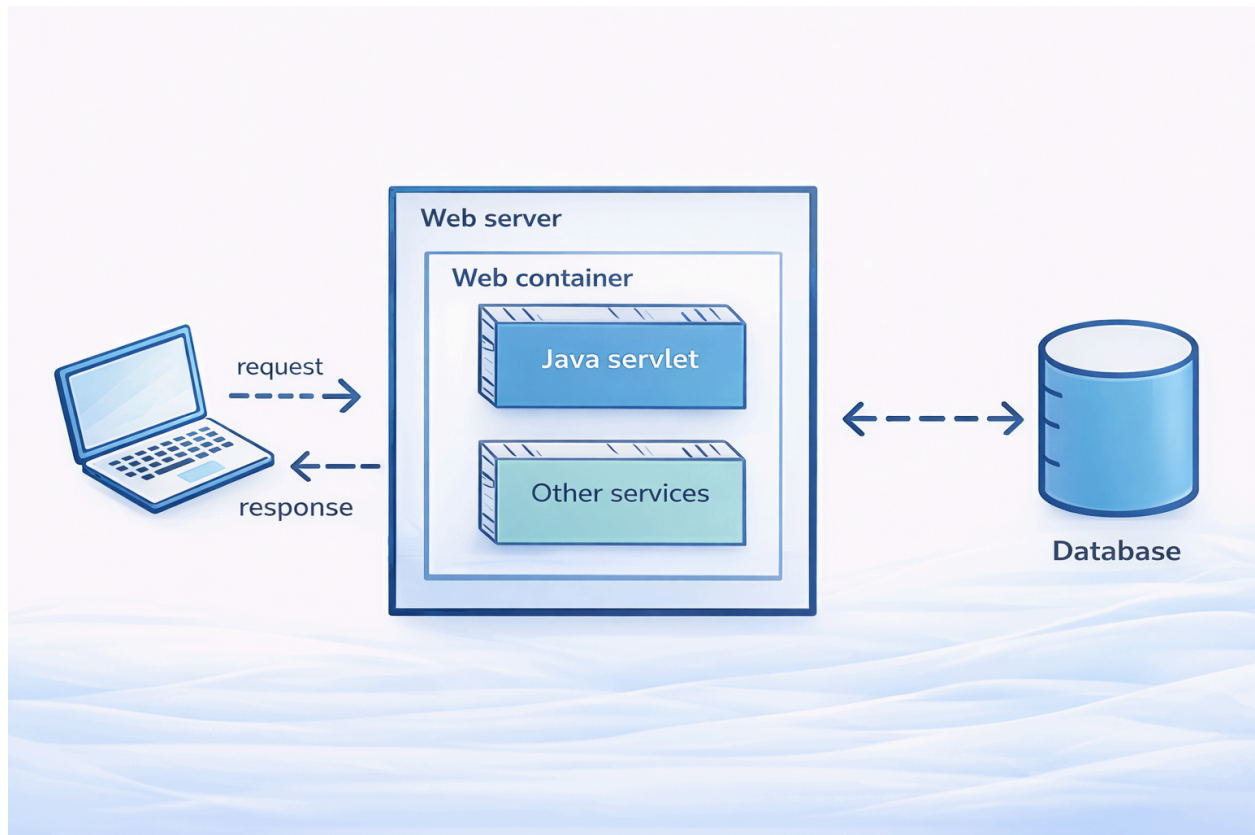
A Web Server is a specialized server that contains only a Web Container. It is designed to handle HTTP requests and serve web-related content efficiently.

- **The Web Container:** Think of this as the "runtime engine" specifically built to manage the life cycle of web-based Java programs.
- **Web Components (The Building Blocks):**
 - **Static Programs:** HTML, CSS, and JavaScript. These provide the structure, style, and client-side behavior of your application.
 - **Dynamic Java Programs:** Servlets and JSPs. These are the server-side programs that process user data, talk to databases, and generate dynamic responses.
- **Best Example:** Apache Tomcat. It is the industry standard for hosting Spring Boot applications and standard Java Web projects

List of webservers

-
1. Tomcat web server
 2. iPlanet web server
 3. Pramathi web server
 4. Java web server

5. Personal web server
6. Resin web server
7. Jetty web server



2. Application Servers (The Enterprise Powerhouse)

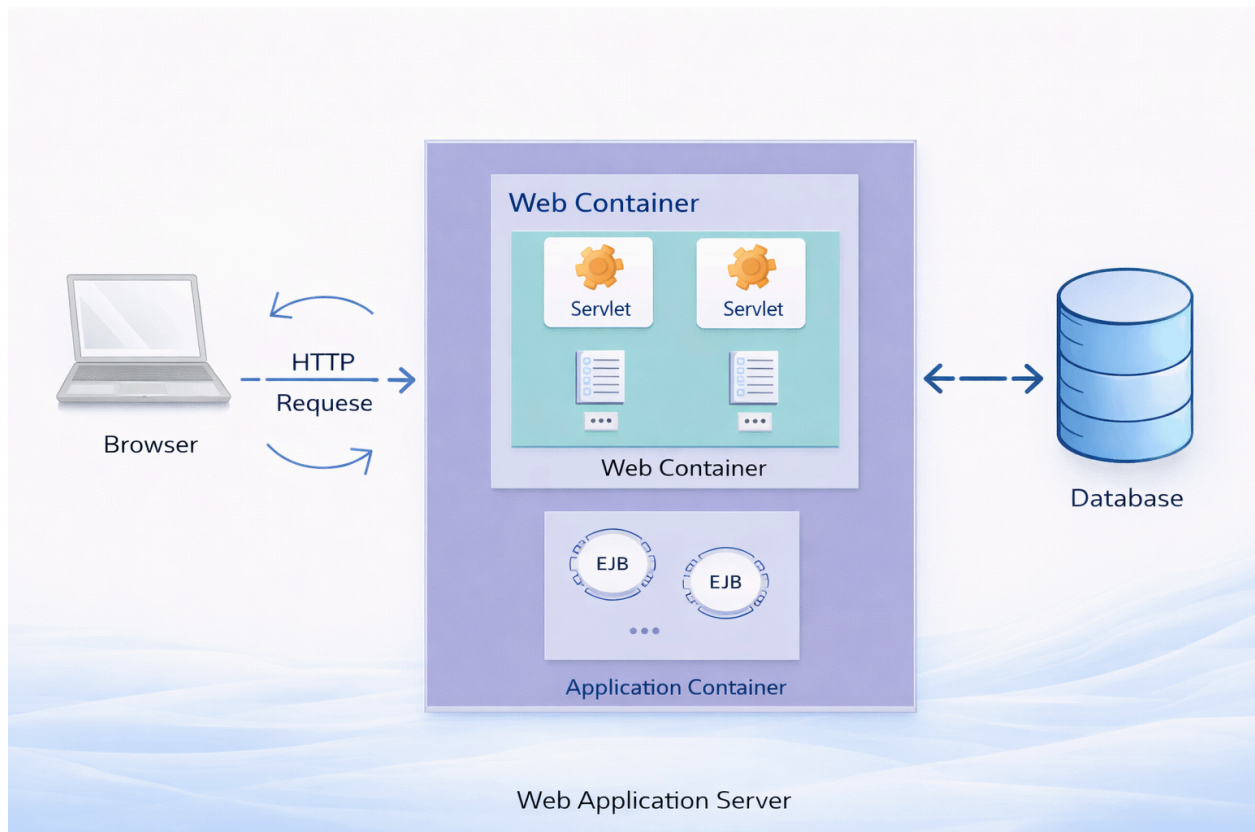
An Application Server is a Superset of a Web Server. It includes everything a Web Server has, but adds heavy-duty tools for complex business needs.

- **The Architecture:** It contains a Web Container PLUS an EJB Container (Enterprise Java Beans).
- **Enterprise Services:** Unlike a simple Web Server, it provides "out-of-the-box" support for:
 - **Distributed Transactions:** Ensuring data integrity across multiple different databases.
 - **Messaging (JMS):** Allowing different applications to communicate reliably through queues.
 - **Resource Pooling:** Managing heavy connections to enterprise systems automatically.

- Best Examples: GlassFish, JBoss (WildFly), and Oracle WebLogic.

List of application server:

1. Web logic application server
2. Web sphere application server
3. JBoss (Wild fly) application server
4. Sun one application server
5. Glass Fish application server



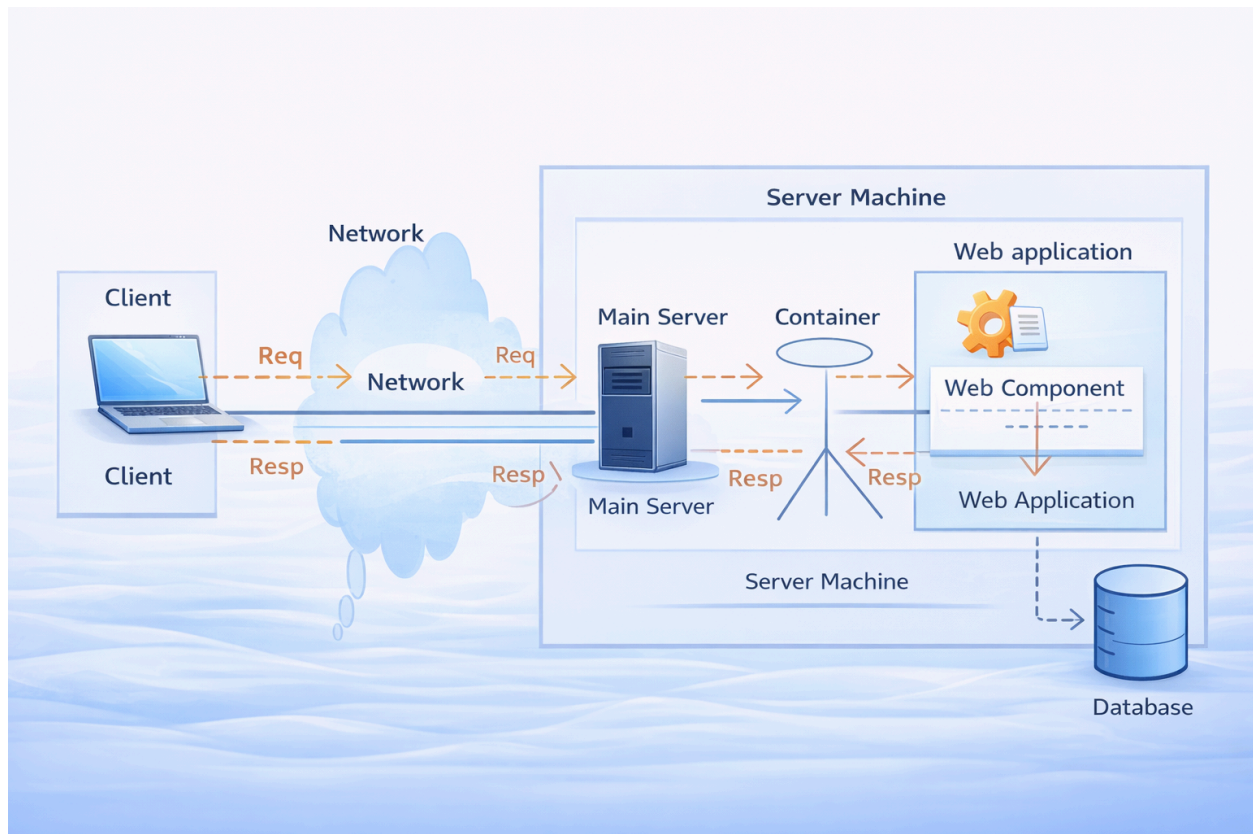
Real-World Comparison: "The Cafe vs. The Factory"

- The Web Server (**The Cafe**): It's fast and perfect for serving "standard orders" (Web pages and APIs). If you just need to show a product list or take a simple order, you use a Cafe.
- The Application Server (**The Factory**): It has a kitchen (Web Container), but it also has a shipping department, a legal team, and a massive supply chain (EJB/Enterprise services). You use this for massive, complex systems like Global Banking or Airline Reservations.

Feature	Web Server	Application Server
Primary Container	Web Container Only	Web + EJB Containers
Key Technologies	Servlets, JSP, HTML, JS	EJB, JMS, JTA, Full Jakarta EE
Main Advantage	Low memory, very fast	High-level enterprise features
When to use?	Most modern Web Apps/APIs	Complex, Distributed Enterprise Systems

When we install server software in computer two component will be installed in our system

1. Main server
2. Container



Q) What is the difference between Main Server and Container?

Ans:

When we submit request from client to server, Main Server will take the request and Main Server will perform the following actions:

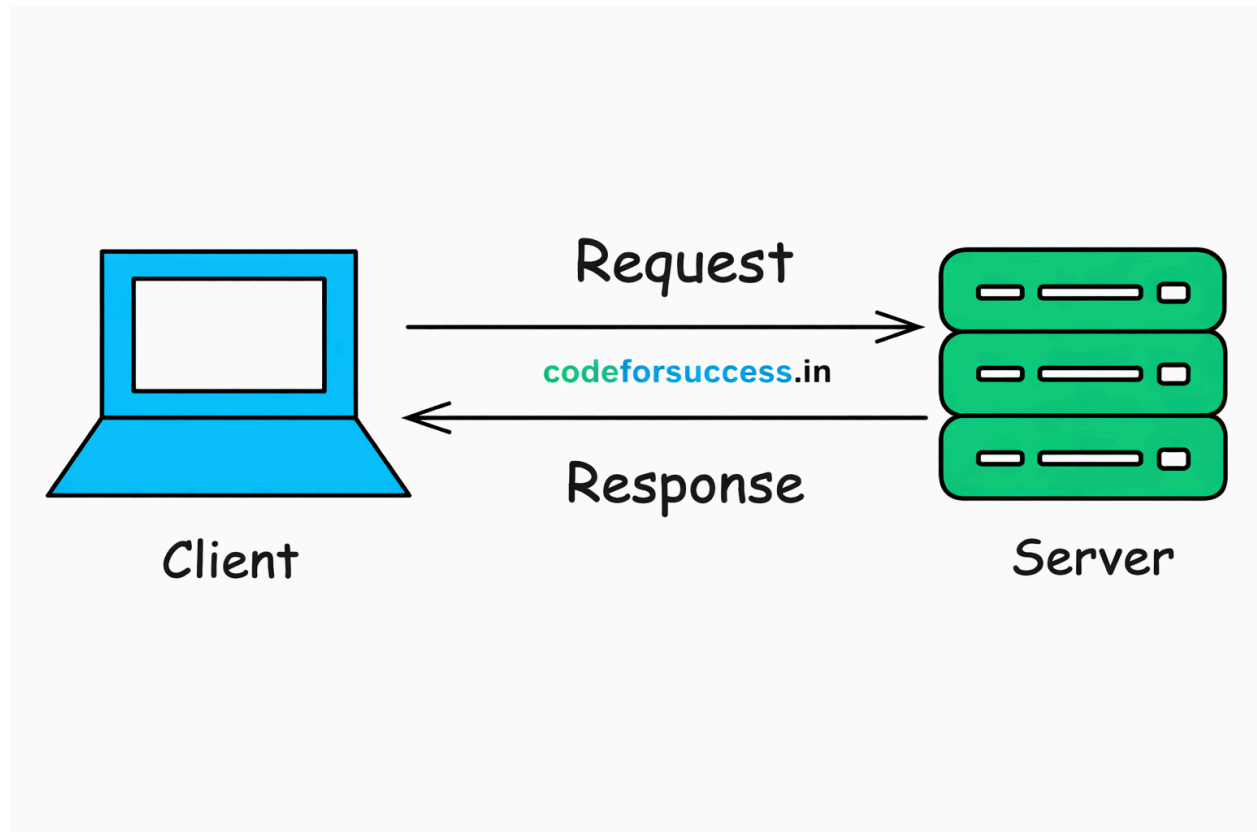
- The Main Server will check whether the request is coming for a static resource or dynamic resource.
 - Static Resource: HTML, CSS, JS

- If the request is coming for the **static resource** then Main Server will identify the respective static resource and Main Server will send the respective static resource as response to the client.
- If the request is coming for any dynamic resource then the Main Server will forward the request to the **Container**.

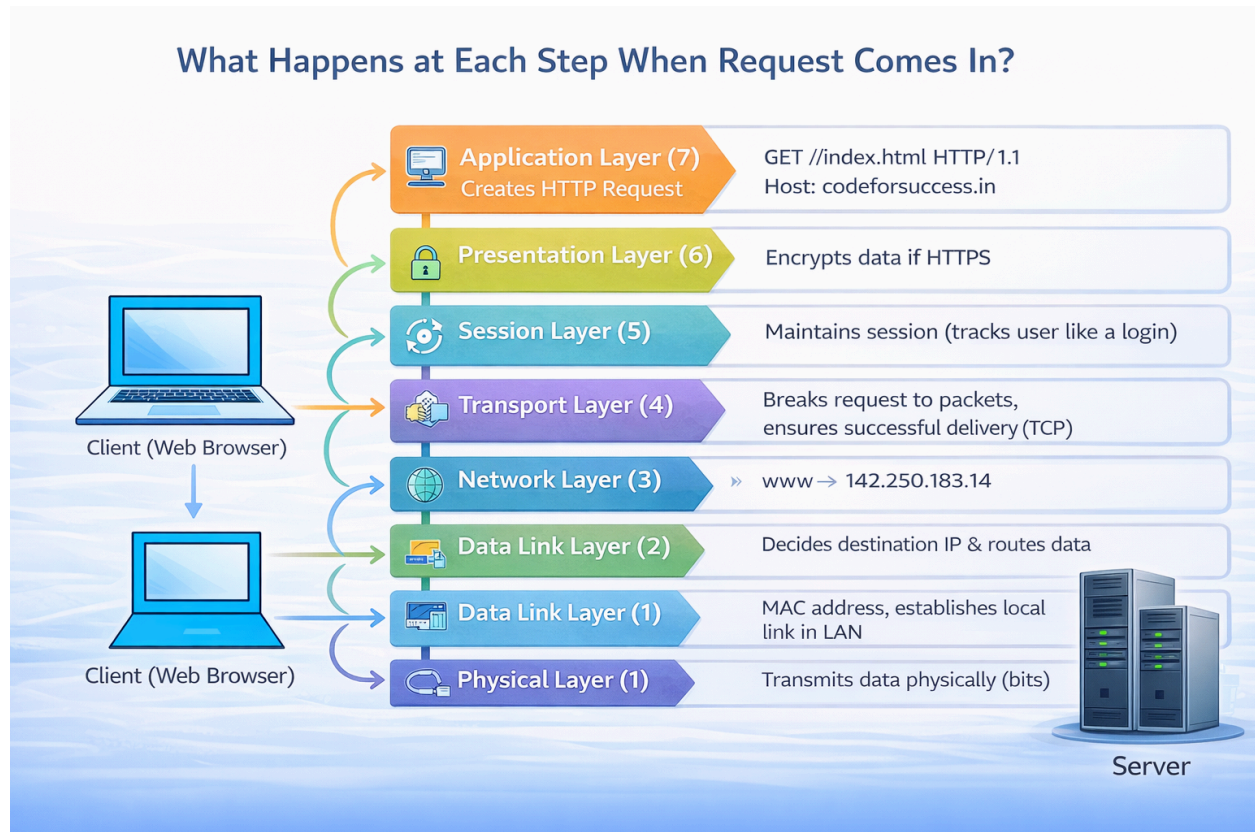
When request is coming to the Container, Container will perform the following actions:

- The container will identify the requested resource at the server machine.
- The container will execute the requested resource at the server machine.
- The container will generate some response.
- The container will submit a response to the respective client.

The Client-Server Model & Request-Response



Fundamentals of Web Communication



Client Request Flow Using OSI Model (HTTP)

When a client sends a request (e.g., <https://codeforsuccess.in>), the data passes through OSI layers as follows:

Application Layer (Layer 7)

- HTTP request is created (GET/POST).
- Headers and request data are added.
- Responsible for communication protocol (HTTP/HTTPS).

Presentation Layer (Layer 6)

- Encrypts data if HTTPS (SSL/TLS).
- Performs data formatting and encoding.
- Ensures secure transmission.

Session Layer (Layer 5)

- Establishes and maintains session.
- Manages session ID and cookies.
- Controls start and termination of communication.

Transport Layer (Layer 4 – TCP)

- Establishes connection using TCP 3-Way Handshake:
 1. SYN
 2. SYN-ACK
 3. ACK
- Breaks data into segments.
- Adds port number (80, 443, 8080).
- Ensures reliable and ordered delivery.

Network Layer (Layer 3 – IP)

- Performs DNS resolution (domain → IP).
- Adds source and destination IP address.
- Routes packets across networks.

Data Link Layer (Layer 2)

- Uses MAC address for local network delivery.
- Transfers frame to next hop (router/switch).

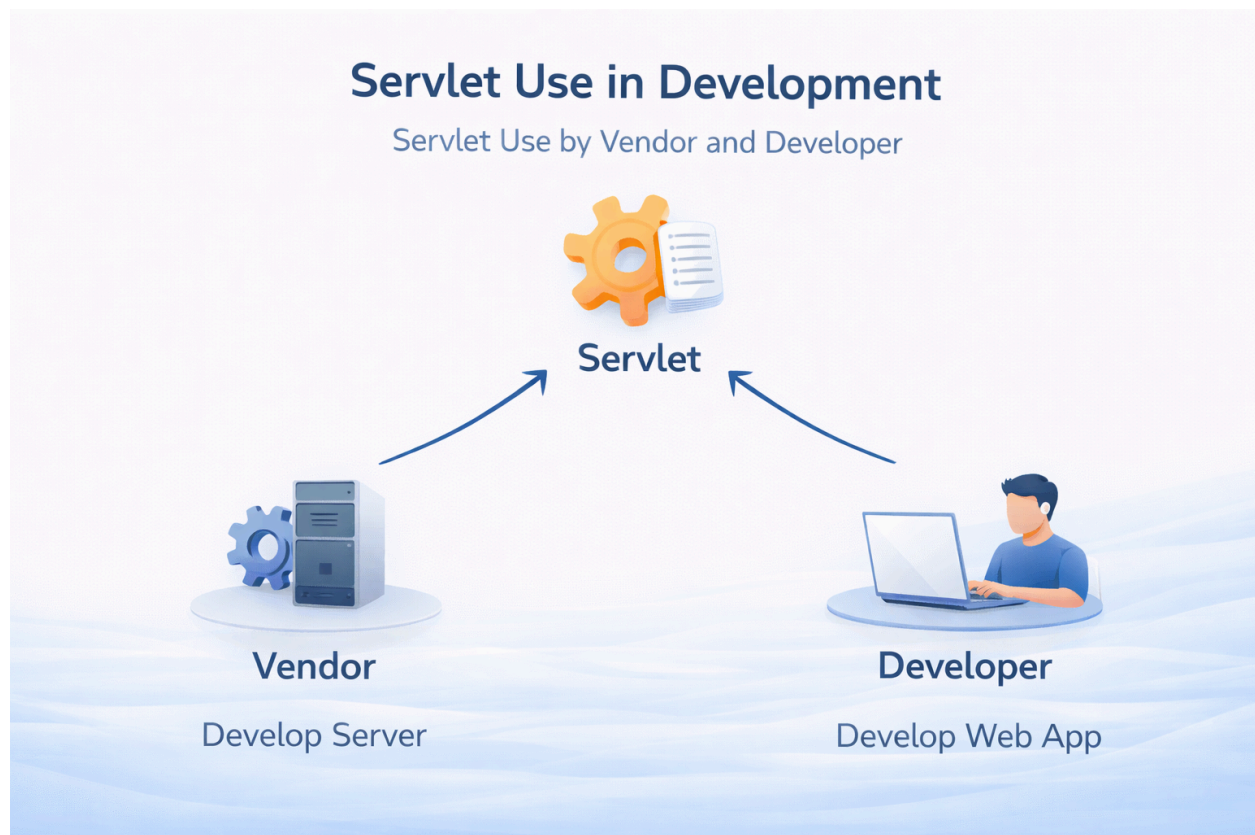
Physical Layer (Layer 1)

- Converts data into bits (0s and 1s).
- Transmits via electrical signals or wireless waves

What is servlet ?

Servlet is a Java **specification** (API) that defines how server-side components should handle HTTP requests and responses. Web container vendors implement this specification to create servlet containers like Tomcat.

Servlet specification used by java programmers to develop web applications.

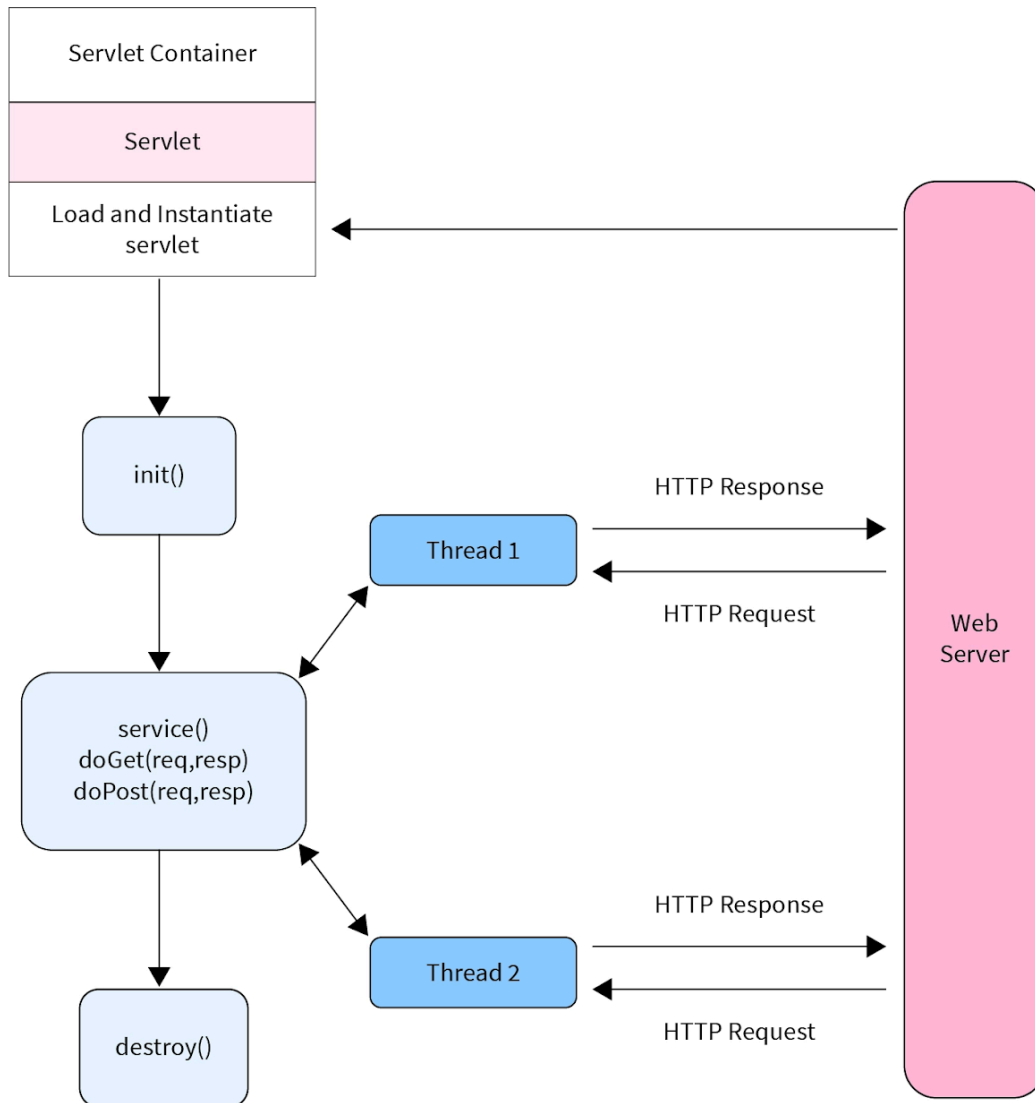


specification : rules + guidelines

So the Question is till 1990 to 1996 how people develop web applications ?

Ans: they use CGI

Servlet Life cycle methods



The Servlet Lifecycle (The "Interview" Favorite)

Students must know that the Container manages the Servlet, not the developer. There are three key phases:

1. **init()**: The "Birth." Called only once when the servlet is first loaded. Used for heavy setup like opening a Database connection.
2. **service()**: The "Work." Called for every request. It decides whether to call **doGet()** or **doPost()**.
3. **destroy()**: The "Death." Called when the server stops. Used to close connections.

"Hello World" Servlet

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.*;

// Annotation mapping: This servlet runs when user visits /greet
@WebServlet("/greet")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        // 1. Set the content type (Telling the browser what's coming)
        resp.setContentType("text/html");

        // 2. Get the writer to "write" the response
        PrintWriter out = resp.getWriter();
        out.println("<h1>Hello Students! Welcome to Advanced Java.</h1>");
    }
}
```